2023 World Conference on Communication & Computing (WCONF)
Raipur, India. July 14-16, 2023

Implementing and Automating Security Scanning to a DevSecOps CI/CD Pipeline

Manohar Marandi
Computer Science and Engineering
Karunya Institute of Technology and Sciences
Coimbatore, India
manoharmarandi@karunya.edu.in

A.Bertia
Computer Science and Engineering
Karunya Institute of Technology and Sciences
Coimbatore, India
bertia@karunya.edu

Salaja Silas
Computer Science and Engineering
Karunya Institute of Technology and Sciences
Coimbatore, India
salaja_cse@karunya.edu

2023 World Conference on Communication & Computing (WCONF) | 979-8-3503-1120-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/WCONF58270.2023.10235015

***Abstract*—With the growing adoption of DevOps and the rise of containerization and Continuous Integration/ Continuous Deployment (CI/CD) in software development life cycle (SDLC) has brought significant changes to the industry. While these methods offer many advantages, they also present unique security challenges, as containerized applications are more susceptible to cyber attacks than traditional deployments, security has become a significant concern. Security scanning is an essential aspect of DevSecOps pipelines, involving the analysis of software images deployed to cloud environments to identify vulnerabilities and mitigate security threats. This study will involve a thorough review of existing literature on containerization and CI/CD security and will analyze current security practices and measures used in containerization-based CI/CD systems. Various tools and techniques have been proposed for implementing and automating image security scanning in DevSecOps pipelines by integrating DAST (Dynamic application security testing) and SAST (Static application security testing) vulnerability scanning. . This research proposes a method for implementing and automating image security scanning using the Snyk and StackHawk tool, which provides a dashboard for SAST and DAST separately for monitoring scanning results and automating vulnerability fixes. The proposed method can be integrated with GitHub, enabling automatic vulnerability scanning and fixing during the build process. The research evaluates the effectiveness of the proposed method by demonstrating the ability of the method to improve the security of DevSecOps pipelines. The findings suggest that the proposed method can enhance the overall security of the application by reducing the time to detect and fix vulnerabilities.**

Keywords—Security Scanning, DevSecOps, Continuous Integration and Continuous Deployment (CICD), Static application security testing(SAST), Dynamic application security testing (DAST)

I. INTRODUCTION

As organizations move towards the adoption of DevOps and agile development approaches, the increasing popularity of containerized Continuous Integration and Continuous Deployment (CI/CD) pipelines have become a vital aspect of modern software development workflows. Containerization has various benefits such as portability, scalability, and consistency across different environments. [19], however security becomes an integral part of the software development process. The incorporation of security into the DevOps pipeline is known as DevSecOps. As shown in fig 1. This approach integrates security into the entire software development life cycle (SDLC) , from planning to deployment, and emphasizes the importance of security in every stage [2]. DevSecOps aims to address the security risks that arise when software is deployed at a rapid pace [6].

Security scanning is a key aspect of the DevSecOps pipeline, and it is critical for discovering vulnerabilities early in the development cycle. Traditional security scanning methods were time-consuming and required manual involvement, leading to delays in the software development process. Implementing and automating security scanning will help minimise the time required to find and patch vulnerabilities, making the process more efficient [5].

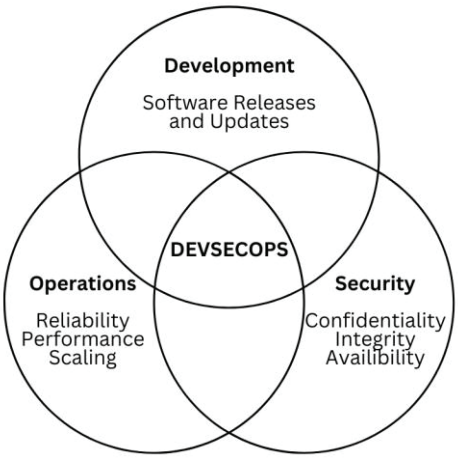

Fig. 1. DevSecOps

To assure the security of containerized CICD pipelines, organisations need to implement both Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) tools[10]. Static Application Security Testing (SAST) is a testing methodology that analyzes source code to find and mitigate security vulnerabilities [9,15]. On the other hand, Dynamic Application Security Testing (DAST) is a method of testing applications while they are operating to discover vulnerabilities [2]. Implementing SAST and DAST tools in the containerized CICD pipeline can help to identify and mitigate security risks throughout the entire software development lifecycle.

The proposed method in this research paper involves implementing and automating security scanning to a DevSecOps CI/CD pipeline. This approach incorporates the use of a dashboard to monitor security scanning using Snyk and StackHawk tools. The dashboard allows for continuous monitoring of vulnerabilities and provides real-time updates on the status of the scanning process [7,8]. The proposed method also includes the ability to automate and fix

vulnerabilities in GitHub, which further reduces the time required to fix vulnerabilities and ensures that vulnerabilities are fixed before code is deployed into production [1].

This research paper aims to investigate the effectiveness of implementing and automating security scanning in a DevSecOps pipeline using the proposed method. The research will evaluate the impact of this approach on the software development process, the efficiency of vulnerability identification and remediation, and the overall security of the software developed. The paper will implement in addition to what was discussed in case studies and also integrate static testing and automate the process [17].

The remainder of this research paper is structured as follows: Section 2 provides a literature review of the existing research on implementing and automating security scanning in a DevSecOps pipeline. Section 3 describes the proposed methodology used in this research. Section 4 presents the Implementation part and followed by section 5 that is Performance. Then Section 6 is Result and analysis of the study. Finally, Section 7 gives the conclusion of the study.

II. Literature Review

The use of DevSecOps practices is becoming increasingly popular in software development environments to ensure security is integrated into the software development process. This makes the security scanning faster, and it can detect security vulnerabilities as early as possible as seen in the paper [2] (Putra and Kabetta) presented a way to integrate DevSecOps within an Agile Software development life cycle utilising GitLab and Docker technologies. The research utilised a hybrid of automated security static testing and dynamic security testing to assure the security. The findings demonstrated that DevSecOps provides a solution to the difficulties of time-consuming build, test, and deployment phases in an Agile SDLC context. Automation of the deployment process lowered the time taken from several hours to 3-4 minutes.

[3] (Chen and Suo) developed a combined operation and maintenance platform of DevSecOps to increase the efficiency of company RnD (research and development) and comprehend security protection automation. DevSecOps is of considerable relevance to ensuring the security of apps and infrastructure. [13] (Sun, Cheng, Qu, and Li) designed and implemented a security test pipeline based on DevSecOps to seamlessly integrate security testing into the software development process. They created a management tool to uniformly manage and show security testing results in reports, and a project management system to build, regress and manage closed-loop security issues. [17] (Rangnau, van Buijtenen, Fransen, and Turkmen) access a way to integrate three automated dynamic testing techniques into a CI/CD pipeline for security testing. They conducted an empirical analysis of the overhead introduced by the automated dynamic testing techniques and identified unique research/technology challenges in DevSecOps, proposing preliminary solutions.

[16] (Rahul, Balabhadruni, Kharvi, and Manu) discussed how DevSecOps provides security while maintaining the speed and agility of DevOps. They demonstrated how open-source tools can be used to implement DevSecOps and provide faster response time and fend off attacks. Implementing and integrating security as part of the pipeline process can create a secure environment and more protected system. In [10] (Díaz, Pérez, Lopez, Mena and Yagüe) they recommend self service cybersecurity monitoring as an enabler to incorporate security practices in a DevOps framework. They underline that the self service monitoring/alerting helps dissolving boundaries between development, operations, and security teams by exposing access to critically important security indicators, which encourages a common culture and continuous development. [7] (Ahmed, Francis) where the authors suggest that integrating security into the DevOps process is essential to ensure secure applications and reduce risk. They argue that DevSecOps is an effective practice for integrating security into the development process. In [12](Zunnurhain, Duclervil) the researchers cover cyber security and several agile software development life-cycle (SDLC) approaches. They focus on DevSecOps and build criteria for a new and effective and efficient DevSecOps approach. The authors suggest that DevSecOps models are more trustworthy than earlier models and that the requirements for a new and successful DevSecOps model were defined. They have designed a website that provides for better security.

The use of automated tools for security and code inspection is another technique to guarantee security is included into the software development process. In [4] (Petrović, Cankar, Luzar,) the authors present a Python-based tool which allows automation of static code analysis and tests for Infrastructure as Code (IaC) scripts. The suggested technology was assessed in different instances when it comes to terraform scripts. They observed that the tool enables automation of code inspection and security checks, therefore boosting the quality of IaC scripts and assuring compliance with standards. in [18] (Hely, Bancel, Flottes, Rouzeyre) the authors present a scan chain integrity detection technique, which supports both automated design flow and IP reuse environment. They argue that the scan path is a profound danger for secure chips, and scanning-based cyber attacks have been shown against DES or AES.

Overall, the literature emphasises the importance of integrating security into the software development process, particularly in Agile SDLC environments, and highlights DevSecOps as an effective practice for achieving this goal. However, there are opportunities for further investigation, which suggests the need for faster and more automated security scanning to detect vulnerabilities at an early stage and Continuous monitoring of. The literature recommends the use of automated security static testing and dynamic security testing tools to ensure code inspection and security checks are performed efficiently and effectively.

III. Proposed Methodology

The proposed methodology for this research focuses on automating image security scanning in a DevSecOps CI/CD pipeline and improving time efficiency. The literature review highlights the importance of integrating security into the software development process and the need for faster and more automated security scanning to detect vulnerabilities at an early stage. Therefore, the proposed technique will focus on combining one of the popular image scanning tools, Snyk and StackHawk, with a CI/CD pipeline to automate Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) scanning throughout the build process. This will help to ensure that code inspection and security checks are performed efficiently and

effectively, ultimately improving the overall security of the software development process.

In this project, a dashboard was created to monitor the security scanning findings generated by Snyk and StackHawk. The dashboard gave a real-time view of the application's security status, allowing developers to easily identify and fix problems. The automation and remediation functionalities of Snyk and StackHawk were connected with GitHub to automate the patching of vulnerabilities discovered during security scanning.

A. Tool Selection

The selection of appropriate tools is crucial for the effective deployment of security scanning in a DevSecOps CI/CD pipeline. In this research, two prominent security scanning tools, Snyk and StackHawk, were selected based on their efficiency in discovering and addressing vulnerabilities in software applications.

Snyk is a cloud-based security scanning tool that specializes in identifying vulnerabilities in open-source libraries and containers. Snyk is easy to connect with CI/CD processes and includes automatic remediation features [7].

StackHawk is a cloud-based dynamic application security testing (DAST) platform that enables security testing. It produces detailed vulnerability reports with specific remedial actions and interacts smoothly with CI/CD processes [8].

The combination of Snyk and StackHawk provides a complete solution to security scanning in a DevSecOps CI CD pipeline. Snyk's focus on SAST and StackHawk's DAST

capabilities allow for a more thorough scan, while StackHawk's automation and remediation features improve efficiency and efficacy. The selection of these technologies was based on their effectiveness, automation capabilities, and simplicity of connection with CI/CD pipelines.

B. DevSecOps Pipeline Design

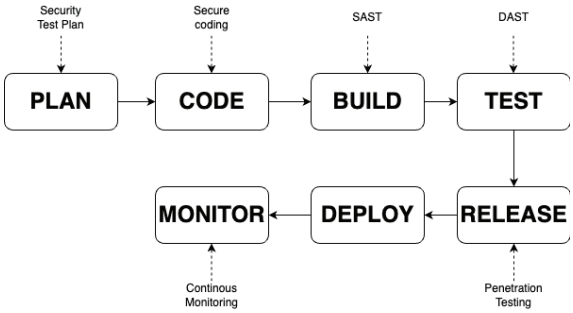

Fig. 2. DevSecOps Pipeline

The term "Plan - Code - Build - Test - Release - Deploy - Monitor" is a commonly used sequence of stages in the software development life cycle (SDLC), known as the "DevOps pipeline." The term "DevSecOps pipeline" refers to a DevOps pipeline that includes security considerations at each stage of the SDLC.

This paper has further specified each stage in the pipeline to include specific security measures. These measures are as follows:

- Plan: The security test plan stage involves planning and designing the security tests that will be carried out at each stage of the pipeline.
- Code: The secure coding stage involves implementing security measures into the code, such as ensuring that code is free of common vulnerabilities like SQL injection and cross-site scripting (XSS).
- Build: The SAST (Static Application Security Testing) stage involves using automated tools to scan the code for vulnerabilities, including both known and potential vulnerabilities. This helps to catch vulnerabilities early in the development process.
- Test: The DAST (Dynamic Application Security Testing) stage involves testing the application in a running environment to simulate real-world attacks and identify any security vulnerabilities that were not caught during the SAST stage.
- Release: The penetration testing stage involves carrying out further testing to identify vulnerabilities that may have been missed in previous stages. This may involve simulating an attack on the application to identify weaknesses that could be exploited.
- Deploy: The deployment stage involves releasing the application to the production environment.
- Monitor: The continuous monitoring stage involves ongoing monitoring of the application to detect and respond to any security incidents that may arise.

IV. IMPLEMENTATION

In this section, a detailed description of the dynamic security testing strategy implementation is provided, along with an implementation diagram (Figure 3). Together with the technologies used to automate security testing, To develop a test environment that can be employed with any CI platform. To build, execute, and distribute the essential apps used in the specified test scenarios, virtualization software programme Docker[21] is used.

With the help of Docker, applications are containerized and run in the CI environment either locally or remotely. Github Actions was chosen as the CI platform for this study, as it offers CI/CD pipelines as a service in a cloud environment.

In order to test our security scanning tool an application is needed. An already vulnerable app was selected “Damn Vulnerable Web Application” [20] as the study target application. The application is a designed vulnerable website and was created for educational and security tool testing purposes. The integration of image scanning tools, specifically Snyk and StackHawk, into the CI/CD pipeline for DevSecOps security scanning involves several steps. First, the respective APIs and integrations of Snyk and StackHawk are configured in the CI/CD pipeline. This may involve setting up authentication, API keys, and other necessary configurations to enable communication between the CI/CD pipeline and the scanning tools.

Next, the image scanning tools are invoked during the appropriate stages of the CI/CD pipeline. For example, Snyk and StackHawk may be triggered to scan container images during the build or deployment stages of the pipeline. The scanning tools analyze the images for known vulnerabilities, security misconfigurations, and other security issues.

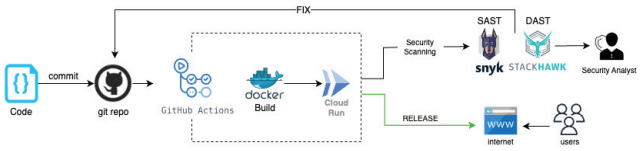

Fig. 3. Architecture

The output from the image scanning tools is then collected and processed to generate meaningful results. This may involve parsing the scan results, extracting relevant information such as vulnerability details, severity levels, and recommended fixes. The processed results are then integrated into respective dashboards, such as the Snyk and StackHawk dashboards, for easy visualization and monitoring of the security status of container images.

The integration of Snyk and StackHawk in the CI/CD pipeline enables automated and continuous security scanning of container images, providing early detection of vulnerabilities and security issues. This allows for timely remediation and ensures that only secure container images are deployed in the production environment, enhancing the overall security posture of the DevSecOps CI/CD pipeline.

V. PERFORMANCE

The performance evaluation of the security scanning process in the DevSecOps CI/CD pipeline, using Snyk and StackHawk, involves assessing the effectiveness and efficiency of the vulnerability detection and reporting mechanism through the following activities:

A. Detection of Vulnerabilities

Snyk and StackHawk are used to scan the Dockerized application for vulnerabilities, security misconfigurations, and other security issues. The scanning process is automated and integrated into the CI/CD pipeline, ensuring that security testing is performed at each stage of the pipeline, from code commit to deployment. shown in fig 4.(a) and 4 (b) are the number of vulnerabilities detected and severity of the vulnerability.

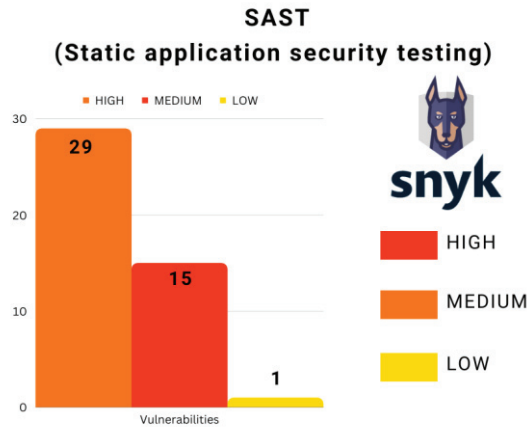

Fig. 4. (a) SAST Report

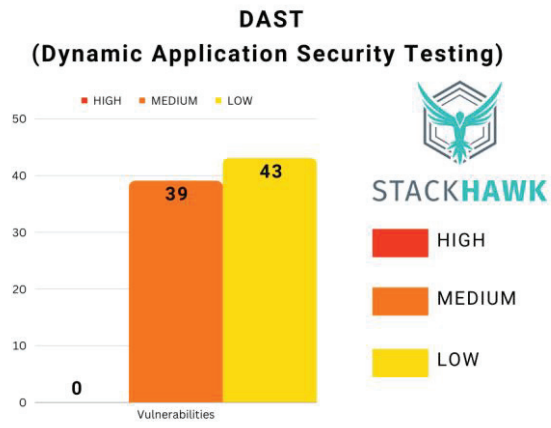

Fig. 4. (b) DAST Report

B. Dashboard Reporting

The results of the security scans are collected and processed, and the vulnerabilities detected by Snyk and StackHawk are displayed in a dashboard. The dashboard provides a clear overview of the vulnerabilities found, their severity levels, and other relevant details. As shown in the fig 5 (a) and 5 (b) below

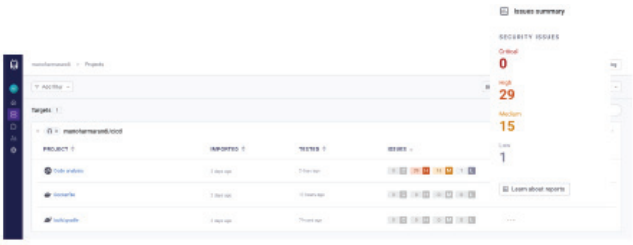

Fig. 5. (a) Snyk Dashboard

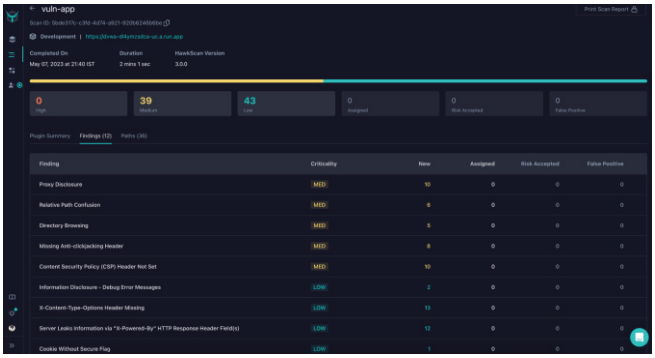

Fig. 5. (b). StackHawk Dashboard

C. Performance Metrics

Performance metrics such as the number of vulnerabilities detected, the severity levels of the vulnerabilities, the time taken for the scanning process are measured to assess the efficiency and effectiveness of the implemented security scanning process.

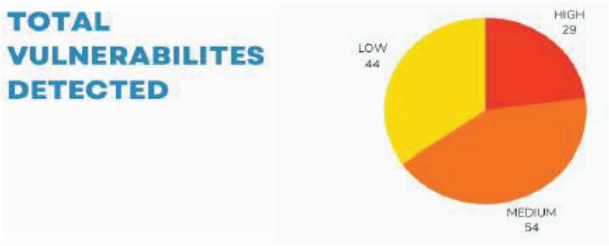

Fig. 6. Total Vulnerabilities detected

TABLE I.

Time taken for scanning process	SAST - Snyk	19 sec
	DAST - Stackhawk	2 min 1 sec

Based on the performance evaluation, the research paper aims to provide insights into the accuracy, effectiveness, and efficiency of the implemented security scanning process using Snyk and StackHawk in the DevSecOps CI/CD pipeline, and how it contributes to enhancing the security of the Dockerized application throughout the CI/CD process.

VI. Result

The implementation and automation of security scanning using Snyk and StackHawk tools in a DevSecOps CI/CD pipeline, along with the integration of image scanning, proved to be an effective approach to improving the security posture of containerized applications. The automated remediation features of Snyk and StackHawk were leveraged to automatically fix vulnerabilities detected during security scanning, saving time and improving efficiency. The integration of image scanning capabilities ensured that container images were thoroughly scanned for vulnerabilities before they were deployed, improving the overall security of the application.

The dashboard developed for this research provided developers with a real-time view of the security state of their apps and container images, enabling for speedy identification and correction of vulnerabilities. This better insight into the security of the programme allows for a more proactive approach to security, rather than depending primarily on reactive vulnerability management.

Overall, the implementation and automation of security scanning applying Snyk and StackHawk technologies, together with the integration of image scanning, improved the security posture of containerized apps in a DevSecOps CI/CD pipeline. The automation and integration of these tools allowed for a more efficient and effective approach to security scanning, minimizing time and increasing the overall security of the application.

VII. Conclusion

To conclude, the implementation and automation of security scanning using Snyk and StackHawk tools, is an effective approach to improving the security posture of containerized applications in a DevSecOps CI/CD pipeline. The selection of Snyk and StackHawk for Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) provide a comprehensive approach to security scanning that ensures vulnerabilities are quickly and effectively remediated.

The automated remediation features of Snyk and StackHawk save time and improve efficiency, while the integration of image scanning ensures that container images are thoroughly scanned for vulnerabilities before they are deployed. The dashboard developed for this research provides developers with a real-time view of the security status of their applications and container images, allowing for quick identification and remediation of vulnerabilities.

VIII. Future scope

In addition to the proposed method for implementing and automating image security scanning using the Snyk and StackHawk tools, there are several avenues for future exploration in the field of securing DevSecOps CI/CD pipelines. extending the proposed method by integrating additional security scanning techniques or tools would enhance the overall effectiveness of vulnerability detection. This could involve incorporating other security testing approaches like fuzz testing, manual code review, or behavioral analysis to provide a more comprehensive assessment of application security. Moreover, scalability and performance are important considerations for large-scale deployments. Future research should explore strategies to optimize resource allocation, parallelize or distribute scanning tasks, and leverage cloud-based infrastructure to enhance the efficiency of the CI/CD pipeline without compromising security. These advancements will contribute to the ongoing efforts to strengthen the security of DevSecOps CI/CD pipelines and mitigate the unique security challenges posed by containerization and continuous deployment methodologies.

REFERENCE

[1] Sonya Moisset, "Open Source Software Security Handbook – Best Practices for Securing Your Projects," freecodecamp, 2023. [Online]. Available: https://www.freecodecamp.org/news/oss-security-best-practices/

[2] A. M. Putra and H. Kabetta, "Implementation of DevSecOps by Integrating Static and Dynamic Security Testing in CI/CD Pipelines," 2022 IEEE International Conference of Computer Science and Information Technology (ICOSNIKOM), Laguboti, North Sumatra, Indonesia, 2022, pp. 1-6, doi: 10.1109/ICOSNIKOM56551.2022.10034883.

[3] T. Chen and H. Suo, "Design and Practice of Security Architecture via DevSecOps Technology," 2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS), Beijing, China, 2022, pp. 310-313, doi: 10.1109/ICSESS54813.2022.9930212.

[4] N. Petrović, M. Cankar and A. Luzar, "Automated Approach to IaC Code Inspection Using Python-Based DevSecOps Tool," 2022 30th Telecommunications Forum (TELFOR), Belgrade, Serbia, 2022, pp. 1-4, doi: 10.1109/TELFOR56187.2022.9983681.

[5] TestingXperts (2022). "Embedding Security Testing in DevOps CI/CD Pipeline" [Online]. Available: https://www.testingxperts.com/blog/security-testing-in-devops

[6] IBM. (2021). "DevSecOps: A Comprehensive Overview." IBM. [Online]. Available: https://www.ibm.com/topics/devsecops

[7] Snyk, "Snyk Security Testing for DevSecOps," Snyk, 2021. [Online]. Available: https://snyk.io/solutions/devsecops/.

[8] StackHawk, "Automate Security Testing in CI/CD," StackHawk, 2021. [Online]. Available: https://www.stackhawk.com/solutions/automated-security-testing/

[9] A. Masood and J. Java, "Static analysis for web service security - Tools & techniques for a secure development life cycle," 2015 IEEE International Symposium on Technologies for Homeland Security (HST), Waltham, MA, USA

[10] J. Díaz, J. E. Pérez, M. A. Lopez-Peña, G. A. Mena and A. Yagüe, "Self-Service Cybersecurity Monitoring as Enabler for DevSecOps," in IEEE Access, vol. 7, pp. 100283-100295, 2019, doi: 10.1109/ACCESS.2019.2930000.

[11] Z. Ahmed and S. C. Francis, "Integrating Security with DevSecOps: Techniques and Challenges," 2019 International Conference on Digitization (ICD), Sharjah, United Arab Emirates, 2019, pp. 178-182, doi: 10.1109/ICD47981.2019.9105789.

[12] K. Zunnurhain and S. R. Duclervil, "A New Project Management Tool Based on DevSecOps," 2019 International Conference on Computational Science and Computational Intelligence (CSCI), Las Vegas, NV, USA, 2019, pp. 239-243, doi: 10.1109/CSCI49370.2019.00049.

[13] X. Sun, Y. Cheng, X. Qu and H. Li, "Design and Implementation of Security Test Pipeline based on DevSecOps," 2021 IEEE 4th Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC), Chongqing, China, 2021, pp. 532-535, doi: 10.1109/IMCEC51613.2021.9482270.

[14] Ö. Şengül, H. Özkılıçaslan, E. Arda, U. Yavanoğlu, İ. A. Doğru and A. A. Selçuk, "Implementing a Method for Docker Image Security," 2021 International Conference on Information Security and Cryptology (ISCTURKEY), Ankara, Turkey, 2021, pp. 34-39, doi: 10.1109/ISCTURKEY53027.2021.9654383.

[15] F. Gauthier, N. Keynes, N. Allen, D. Corney and P. Krishnan, "Scalable Static Analysis to Detect Security Vulnerabilities: Challenges and Solutions," 2018 IEEE Cybersecurity Development (SecDev), Cambridge, MA, USA, 2018, pp. 134-134, doi: 10.1109/SecDev.2018.00030.

[16] Rahul, Balabhadruni, Prajwal Kharvi and Monto Manu. "Implementation of DevSecOps using Open-Source tools." International Journal of Advance Research, Ideas and Innovations in Technology 5 (2019): 1050-1051.

[17] T. Rangnau, R. v. Buijtenen, F. Fransen and F. Turkmen, "Continuous Security Testing: A Case Study on Integrating Dynamic Security Testing Tools in CI/CD Pipelines," 2020 IEEE 24th International Enterprise Distributed Object Computing Conference (EDOC), Eindhoven, Netherlands, 2020, pp. 145-154, doi: 10.1109/EDOC49727.2020.00026.

[18] D. Hely, F. Bancel, M. . -L. Flottes and B. Rouzeyre, "A secure Scan Design Methodology," Proceedings of the Design Automation & Test in Europe Conference, Munich, Germany, 2006, pp. 1-2, doi: 10.1109/DATE.2006.244019.

[19] S. Singh and N. Singh, "Containers & Docker: Emerging roles & future of Cloud technology," 2016 2nd International Conference on Applied and Theoretical Computing and Communication Technology (iCATccT), Bangalore, India, 2016, pp. 804-807, doi: 10.1109/ICATCCT.2016.7912109.

[20] github. "DAMN VULNERABLE WEB APPLICATION" [Online]. Available : https://github.com/digininja/DVWA

[21] docker. "Docker Overview" [online]. Available : https://docs.docker.com/get-started/overview/